

UE 7 : Mobility in Smart Cities

Philippe Canalda

Philippe.Canalda@univ-fcomte.fr

- Three main objectives are :
 - *Made* François Spies : Indoor/Outdoor Positioning, mobility and tracking using radio-based technologies (wifi, lora, Bluetooth, 3G/4G/5G, satellite, ...)
 - *Current* Philippe Canalda : Reactive/Real-time organization of mobility (optimization, prediction) with modelling&methodology, pragmatic implementations
 - *To Be Done* Oumaya Baala : Mobility model and IA based analysis

Reactive/Real-time organization of mobility (optimization, prediction) with modelling&methodology, pragmatic implementations

- Course/TD/TP/mini-project = 4/6/8/18

Reactive/Real-time organization of mobility (optimization, prediction) with modelling&methodology, pragmatic implementations

- Real problem exampleS modelling and resolution:
 - Dynamic Louage
 - ...
- Pragmatic algorithms, Graphs, orders, modelling, ...
- Exercises by group of 2 or 3
- A project by group of 2 or 3

Program of the day 23/09/2025

- 1h of presentation of Louage problem : definition, specification and solving.

Program of the day 23/09/2025

- 1h of presentation of Louage problem : definition, specification and solving.
- 1h15 of course introducing orders and combinatorial algorithm by enumeration : greedy algorithm, enumeration with or without orders, enumeration with cut and price (when the solution computed will not improve the best results already computed), a recursive algorithm also known as dynamic programming algorithm with or without cut and price.

Program of the day 23/09/2025

- 1h of presentation of Louage problem : definition, specification and solving.
- 1h15 of course introducing orders and combinatorial algorithm by enumeration : greedy algorithm, enumeration with or without orders, enumeration with cut and price (when the solution computed will not improve the best results already computed), a recursive algorithm also known as dynamic programming algorithm with or without cut and price.
- 2h30 of practical work and project presentation

Case study of an organization / mobility optimization project

Theoretical supports of orders, properties,
graphs to model complex mobilities and to
solve exactly the problems of organization
of mobilities

References

- **Reference books (main)**

Authors

Title

- Thomas Cormen,

INTRODUCTION TO ALGORITHMS, 3rd edition,
MIT PRESS (En & Fr)

- Charles Leiserson
et Ronald Rivest,

- Faure, Lemaire, Picouleau

Précis de Recherche
Opérationnelle, 6ème édition Dunod ;

- **Required materials :**

- The 2 books of Cormen, Leiserson and Rivest
et Faure, Lemaire et Picouleau

- **Available materials :**

- Slides + interactive explanations
 - One graph and exact optimisation support
 - (2 addings on linear programming, available in Fr on demand)

Evaluations

- A final & theoretical exam 50%
- Practice and exercices 20%
- A short project per group of 2 (to 3, or alone also accepted) 30%
 - Either you select to implement (In Python, Java, PHP/JS, or C++) one of the 4 algorithms (Louage, On Demand Transportation, Dynamic Carpooling, Carpooling++),
 - Or you choose the Positioning and Optimizing Mobility Application, to be detailed thereafter.

A short project per group of 2 (to 3, or alone also accepted) 30%

- A mobile application (preferably Android) or a web-based application
 - which maps student locations (homes, classrooms, places of life or activity, meeting places, your car parks, etc.),
 - which calculates the OD matrix of these spaces (places, distances, time, routes,...).

A short project per group of 2 (to 3, or alone also accepted) 30%

- A mobile application (preferably Android) or a web-based application
 - which maps student locations (homes, classrooms, places of life or activity, meeting places, your car parks, etc.),
 - which calculates the OD matrix of these spaces (places, distances, time, routes,...).
- Tree functions to conceive and implement

A short project per group of 2 (to 3, or alone also accepted) 30%

- A mobile application (preferably Android) or a web-based application
 - which maps student locations (homes, classrooms, places of life or activity, meeting places, your car parks, etc.),
 - which calculates the OD matrix of these spaces (places, distances, time, routes,...).
- Tree functions to conceive and implement :
 - the main function is to determine the place.x and the time window where a group of (registered) friends can meet at the earliest. (rated out of 14)

A short project per group of 2 (to 3, or alone also accepted) 30%

- A mobile application (preferably Android) or a web-based application
 - which maps student locations (homes, classrooms, places of life or activity, meeting places, your car parks, etc.),
 - which calculates the OD matrix of these spaces (places, distances, time, routes,...).
- Tree functions to conceive and implement :
 - the main function is to determine the place.x and the time window where a group of (registered) friends can meet at the earliest. (rated out of 14),
 - the 2nd function is to generate the routes of each friend (rated out of 18)

A short project per group of 2 (to 3, or alone also accepted) 30%

- A mobile application (preferably Android) or a web-based application
 - which maps student locations (homes, classrooms, places of life or activity, meeting places, your car parks, etc.),
 - which calculates the OD matrix of these spaces (places, distances, time, routes,...).
- Tree functions to conceive and implement :
 - the main function is to determine the place.x and the time window where a group of (registered) friends can meet at the earliest. (rated out of 14),
 - the 2nd function is to generate the routes of each friend (rated out of 18),
 - and the 3rd function is to follow the friends step by step to signal the delay or the advance of this mission (you will have 20 / 20 if you can!).

A short project per group of 2 (to 3, or alone also accepted) 30%

- A mobile application (preferably Android) or a web-based application
 - which maps student locations (homes, classrooms, places of life or activity, meeting places, your car parks, etc.),
 - which calculates the OD matrix of these spaces (places, distances, time, routes,...).
- Tree functions to conceive and implement :
 - the main function is to determine the place.x and the time window where a group of (registered) friends can meet at the earliest. (rated out of 14)
 - the 2nd function is to generate the routes of each friend (rated out of 18),
 - and the 3rd function is to follow the friends step by step to signal the delay or the advance of this mission (you will have 20 / 20 if you can!).
- The right modeling and shaping of the conduct of this project can give you an 8/20.

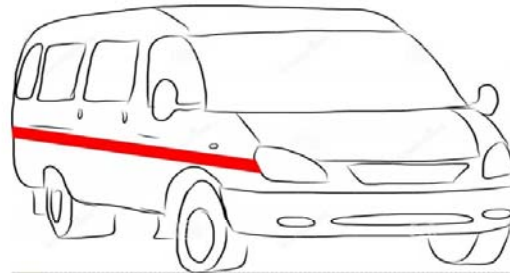
Detail of the mini-project

- Write the subject (with illustrations),
- take note of the Specifications, guidelines
- Describe the necessary data (writings and illustrations), essential functionality and any underlying functions required
- Build the architecture of the application by projecting the known actors, the functions in the scenario (s) constituting the priority Specifications

First Digital Tunisian Louage's Transportation Solution



First Digital Tunisian Louage's Transportation Solution



Realized by:

❖ Mohamed HASSINE
mohamedhassinef35@gmail.com

❖ Philippe CANALDA
philippe.canalda@femto-st.fr

❖ Idriss HASSINE
drisshassine@yahoo.fr



First Digital Tunisian Louage's Transportation Solution

- Idriss Hassine is a Master 2 student promotion 2014-2015
- Mohamed Hassine is a master 2 student, promotion 2017-2018

(Mohamed and Idriss are brothers).

First Digital Tunisian Louage's Transportation Solution

- The presentation to be downloaded separately.

1. Techniques, Methods and Tools: An Overview.

- **<Exercices > :**
 - **"Makes change",**
 - **Given, on the one hand, a list of coins (ordered or not) and given an amount of change to be returned**
 - n **Calculate the combinations of parts to be returned**
 - » **The best**
 - » **All**
 - » **A**
 - **Discussion**
 - **"The eight queens"**
 - **"Resource allocation"**

Interaction - Resource allocation

- A set of student groups or lists, with the number
 - Diagram
 - A list ordered: 3, 15, 17, ..., 127, ...
 - Or a list not ordered: 17, 3, 127, 15, ...
- Set or list of classroom with capacities
 - 20, 30, 30, 70, ...
- Problem: find, if it exists, an assignment or we make sure that each group will have a classroom assignment
 - Rmk : problems can always become more complex, such as: finding the assignment that:
 - n Minimize the occupation rate
 - n Or maximize it

Interaction - Resource allocation (following)

- Example : GoS = 126, 15, 3, 84, 45
- GoC= 30, 90, 89, 30, 140
- The basic method :
 - Browse GoS from left->right
and for each array's item do
browse GoC from l->r
looking for a free assignment
- This method fails: 126 to 140, 15 to 30, 3 to 90, 84 to 89, but we can no longer assign 45
- Note: a solution is more easily found when ordering the manipulated elements. For example the increasing or decreasing choice of ordering the list of groups. Likewise for the list of classes and their capacities.

"Makes change"

- Given, on the one hand, a list of coins (ordered or not) and given an amount of change to be returned
 - n We give ourselves an ordered List $L = \{5, 2, 1, 0.5, 0.2, 0.1, 0.05\}$ with $L[0] = 5, \dots L[6] = 0.05$
 - n An amount $m = 12.35$ euros of change to be returned
- Calculate the combinations of parts to be returned
 - The best
 - All
 - A
 - Discussion

"Make change": towards a solution

n $L = \{5, 2, 1, 0.5, 0.2, 0.1, 0.05\}$ with $L[0] = 5, \dots L[6] = 0.05$ $m = 12.35$
euros

- Calculate the combinations of parts to be returned

- The best, all, one, talk

n Idea: calculate

- » i) the number of 5 euro coins to be returned out of 12.35 $\Rightarrow$ 2,
 - » ii) the remainder $\Rightarrow 12.35 - 2 * 5 = 2.35$,
 - » iii) go to the next part and repeat steps i) and ii) until the end of the list or until the remainder is zero

"Make change": towards a solution

n $L = \{5, 2, 1, 0.5, 0.2, 0.1, 0.05\}$ with $L[0] = 5, \dots L[6] = 0.05$ $m = 12.35$ euros
calculate

- » i) the number of 5 euro coins to be returned out of 12.35 $\Rightarrow 2$,
- » ii) the remainder $\Rightarrow 12.35 - 2 * 5 = 2.35$,
- » iii) go to the next part and repeat steps i) and ii) until the end of the list or until the remainder is zero

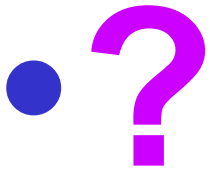
- Iterate the example,
- Automate, design your program
- analyze the method:
 - accuracy / proof, observation of the solution / generalization, improvement / complexity

Iterate the example

n $L = \{5, 2, 1, 0.5, 0.2, 0.1, 0.05\}$ with $L[0] = 5, \dots L[6] = 0.05$ $m = 12.35$
euros calculate

- » i) the number of pieces to be returned on $m = 12.35$, $i = 0$,
 $L[0] = 5$
 $\text{quotient} = 2 = 12.35 / 5 = m / L[i]$
- » ii) the remainder $m = 2.35 = 12.35 \% 5 = m \% L[i]$,
- » iii) go to the next part
and repeat steps i) and ii)
until the end of the list or until the remainder is zero
 $i++$

Automate, design your program



(Iterative, recursive, parallel program, ...)

Then analyze the method:

- **accuracy / proof, observation of the solution / generalization, improvement / complexity**

Example solution

n $L = \{5, 2, 1, 0.5, 0.2, 0.1, 0.05\}$ with $L[0] = 5, \dots L[6] = 0.05$
 $m = 12.35$ euros
calculate

```
» While ( $m \neq 0$ ) and ( $i < L.length$ ) do
    // loop invariant
    // we have calculated the optimal occurrences of the parts located
    // at row  $0..i-1$ 
    Print  $m / L[i] + \text{"Coins of"} + L[i] + \text{"euros"}$ 
     $m = m \% L[i]$ 
end while
// exit conditions of the loop:
//  $m == 0$  or  $i \geq L.length$ 
```

analyze the method:

accuracy / proof, observation of the solution / generalization,
improvement / complexity

analyze the method:

- accuracy / proof,
 - Already done
- observation of the solution / generalization,
 - Calculate the best solution if it exists. That matches when / because
 - The list is ordered in descending order
 - n Obvious counterexample
 - The elements located at ranks $l + 1$ and beyond are generators (to constitute any amount, a fortiori $\leq$ that of $L[i]$)
 - n $L = \{5, 2, 1.5\}$ $m = 9.5$

Work to do

- TP “makes change”: with L and m given as an example
- i) iterative greedy solution
- li) iterative solution of all solutions with decreasing L and also the combinations of the selected part are treated in decreasing order
- lii) stop with the 1st good solution. Is this the best?
- lv) recursive approach

analyze the method:

- Notes:
- Greedy method which calculates the optimal when the best local choice (made at index i) leads to the best global choice whatever are the next indices
 - See next course on graphs and greedy optimization methods

analyze the method:

- Calculating a solution? **Calculating all the solutions?**

n ?

n Often it is a question of apprehending a recursive approach

Example of alternative solution

n New steps:

- » represent the solution
- » Generate solutions recursively
- » Treat the stop condition

n $L = \{5, 2, 1, 0.5, 0.2, 0.1, 0.05\}$ with $L[0] = 5, \dots, L[6] = 0.05$
m = 12.35 euros, S_fifo = {}
calculate

n while (m <> 0) and (i <= L.length) do
 // INVARIANT
 // we calculate all the occurrences of the coins located
 // at row 0..i-1
 // we store these occurrences in a list FIFO
 for j ranging from 0 to m / L[i] do
 S_fifo.add(j) RelaunchCalculation (L, m- j * L[i], i + 1, S_fifo)
 S_fifo.subtractlasteltinserted
 end for

Methodological recipe

- To solve a problem, there are several recipes:
 - The first is to find an algorithm solving a version that simplifies the solution. This can be obtained
 - by generalizing the problem
 - n Do not limit the number of occurrences of parts
 - By simplifying it from a combinatorial point of view
 - n Order the list of coins in descending order of coin values
 - Then, it is always easier to add constraints such as limiting the number of occurrences of parts
 - But it will be less so to take into account an unordered list that we can order to come down to the problem that we will have been able to solve or to see the solution

Example of alternative solution

n **New steps:**

- » **represent the solution**
- » **Generate solutions recursively**
- » **Treat the stop condition**

n **L = {5, 2, 1, 0.5, 0.2, 0.1, 0.05} with L [0] = 5,... L [6] = 0.05**
m = 12.35 euros,
S_fifo = {}
calculate

» **If m == 0**

// solution found

print S_fifo

else

// m <> 0

if i >= L.length

// no solution

else // m <> 0 and i <= L.length

// we continue

// we calculate all the occurrences of the parts located

// at rank 0..i-1 and we store these occurrences in a FIFO list

for j ranging from 0 to m / L [i] do

S_fifo.add (j)

RestartCalculate (L, m- j * L [i], i + 1, S_fifo)

S_fifo.subtractlasteltinserted

end for

- Program
 - implement the recursive program
 - Apply the method to the 8 queens exercise
 - Present
 - ordered structures
 - Applications of trellises and
 - of Boolean algebra
 - Exercises

Work at home

- 4 school practical exercises
 - 1 = implement “makes change” algorithm in greedy way
 - 2 = enumerate all generations
 - 3 = enumerate only solutions that improve the best solutions computed until now (with cut implementation)
 - 4 = recursive form (with cut implementation)

Basic supports in java, C ++, C will be offered for download



```
def make_change(amount):  
    change= [5, 2, 1, 0.5, 0.2, 0.1, 0.05]  
    changeCounter = [0, 0, 0, 0, 0, 0, 0]  
    print("Change count -> ")  
    # amount = float(amount)  
    for i, j in zip(change, changeCounter):  
        if amount >= i:  
            j = amount // i  
            amount = amount - j * i  
            amount = round(amount,2)  
            print(i, " : ", j)  
  
make_change(12.35)
```

this is the best one (modifié)



```
coins = [5, 2, 1, 0.5, 0.2, 0.1, 0.05]
m = 12.35
i = 0

print("Coins disponibles : " + str(coins))
print("Money to be returned : " + str(m))

while(m != 0 and i < len(coins)):
    return_coin = int(round(m, 2)/coins[i])
    print("We return " + str(return_coin) + " of " + str(coins[i]) + " euros")
    m = m - round(return_coin * coins[i], 2)
    print("The money we need to return is equals to " + str(round(m, 2)) + " euros \n")
    i += 1
```

I have that

```
'
Coins disponibles : [5, 2, 1, 0.5, 0.2, 0.1, 0.05]
Money to be returned : 12.35
We return 2 of 5 euros
The money we need to return is equals to 2.35 euros

We return 1 of 2 euros
The money we need to return is equals to 0.35 euros

We return 0 of 1 euros
The money we need to return is equals to 0.35 euros

We return 0 of 0.5 euros
The money we need to return is equals to 0.35 euros

We return 1 of 0.2 euros
The money we need to return is equals to 0.15 euros

We return 1 of 0.1 euros
The money we need to return is equals to 0.05 euros

We return 1 of 0.05 euros
The money we need to return is equals to -0.0 euros
```




Enguerrand BOITEL 25/11/2020

```
We need 2 of 5 euros  
We need 1 of 2 euros  
We need 1 of 0.2 euros  
We need 1 of 0.1 euros  
We need 1 of 0.05 euros
```

```
while(m != 0 and i < len(coins)):  
    coin_to_return = int(round(m, 2)/coins[i])  
    if(coin_to_return != 0):  
        print("We need " + str(coin_to_return) + " of " + str(coins[i]) + " euros")  
    m = m - round(coin_to_return * coins[i], 2)  
    #print("The money we need to return is equals to " + str(round(m, 2)) + " euros \n")  
    i += 1
```

This is my code

No, Python



Abdessamad HAMDAOUI 25/11/2020

```
L=[5,2,1,0.5,0.2,0.1,0.05]
m=12.35
R=[]
i=0
for i in range(len(L)):
    q=m/L[i]
    R.append(int(q))
    m=m%L[i]
    print('we need',R[i],"coins of",L[i])
```

```
we need 2 coins of 5
we need 1 coins of 2
we need 0 coins of 1
we need 0 coins of 0.5
we need 1 coins of 0.2
we need 1 coins of 0.1
we need 0 coins of 0.05
```

python solution solution



Kwabena Bamfo 25/11/2020

Greedy implementation with timer

```
1- def make_change(amount):
2-     L = [5,2,1,0.5,0.2,0.1,0.05]
3-     res = []
4-
5-     for i in range(len(L)):
6-         number = amount/L[i]
7-         amount = amount % L[i]
8-         res.append(int(number))
9-
10-    if res[i] != 0:
11-        print(L[i], ":", res[i], "coins")
12-
13-
14-    import time
15-    start = time.time()
16-    make_change(12.35)
17-    end = time.time()
18-    print("seconds:",end - start)
```

```
5 : 2 coins
2 : 1 coins
0.2 : 1 coins
0.1 : 1 coins
seconds: 0.0001110305786132812
```



JoshuaShakya 25/11/2020

```
import time

def best_solution(amount):

    notes = [5, 2, 1, 0.5, 0.2, 0.1, 0.05]
    remainder_amount=[]

    noteCounter = [0, 0, 0, 0, 0, 0, 0]

    print("Best solution is:\n ")
    # amount = float(amount)
    for i in range(len(notes)):
        j = amount /notes[i]
        remainder_amount.append(int(j))
        amount = round(amount%notes[i],2)
        if(remainder_amount[i]!=0):
            print(remainder_amount[i], ' coin(s) of ', notes[i],

    start_time=time.time()
    best_solution(12.35)
    end_time= time.time()-start_time

    print('\nTime of execution is: {} seconds'.format(end_time))
```

Best solution is:

```
2 coin(s) of 5 euros
1 coin(s) of 2 euros
1 coin(s) of 0.2 euros
1 coin(s) of 0.1 euros
1 coin(s) of 0.05 euros
```

Time of execution is: 0.000682830810546875 seconds

Ishan Gautam a répondu 25/11/2020



Abdessamad HAMDAOUI 25/11/2020

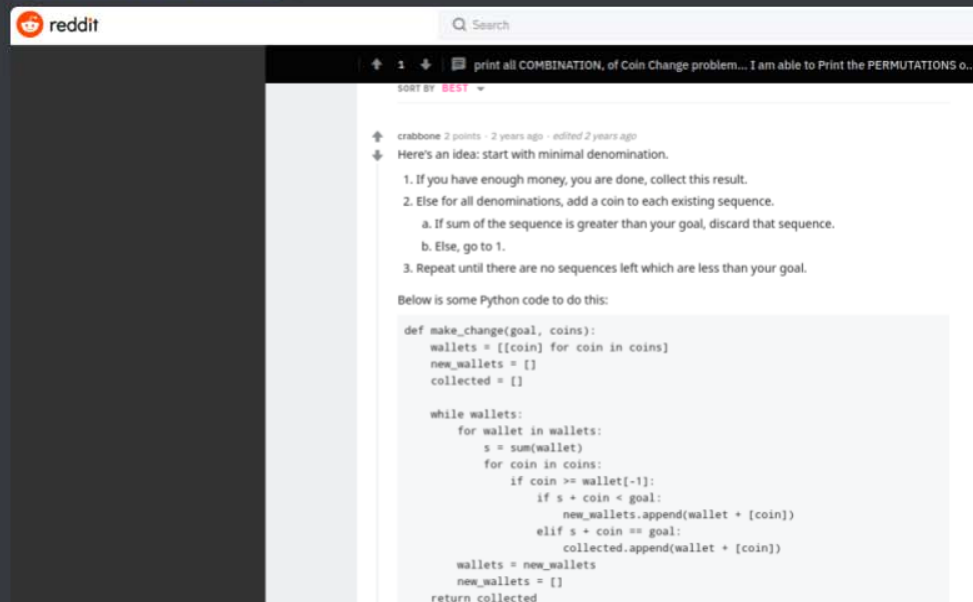
```
import time
start=time.time()
L=[5,2,1,0.5,0.2,0.1,0.05]
m=12.35
R=[]
for i in range(len(L)):
    q=m/L[i]
    if 0.99999<q<1:
        q=1
    R.append(int(q))
    m=m%L[i]
    print('we need',int(R[i]),"coins of",L[i])
end=time.time()
print('Execution time :',end - start)
```

```
we need 2 coins of 5
we need 1 coins of 2
we need 0 coins of 1
we need 0 coins of 0.5
we need 1 coins of 0.2
we need 1 coins of 0.1
we need 1 coins of 0.05
Execution time : 0.0011174678802490234
```



Victor NAGY 25/11/2020

@Julien LECOQ , en fait ils trouvent tous leur code sur Reddit, Stackoverflow et compagnie, je vais bosser direct le projet





Setareh Nouri 25/11/2020

```
import time
def bestSolution(x):
    List = [5, 2, 1, 0.5, 0.2, 0.1, 0.05]
    cash=x
    R=[]
    for j in range (len(coins)):
        q= cash/coins[j]
        R.append(int(q))
        cash = cash% List[j]
        print(List[j],":", R[j])

    startTime = time.time()
    bestSolution(12.35)
    endTime= time.time()
    print(endTime-startTime)

5 : 2
2 : 1
1 : 0
0.5 : 0
0.2 : 1
0.1 : 1
0.05 : 0
0.00047588348388671875
```



Julien LECOQ 25/11/2020

```
1 from time import perf_counter
2
3 coins = (5, 2, 1, 0.5, 0.2, 0.1, 0.05)
4 money_to_return = 12.35
5
6 def make_change(coins, money_to_return):
7     """
8     Print the best solution of the make change problem.
9     """
10
11     print(f'For the best solution, we need:\n')
12
13     # Loop through every coins.
14     for coin in coins:
15         number_of_coins = int(money_to_return / coin) # Compute the number of a specific coin required.
16         money_to_return = money_to_return % coin # Update the amount of money to return.
17
18         # Print the number of a specific coin required.
19         if number_of_coins != 0:
20             print(f'{number_of_coins} coins of {coin}.')
21
22     # Perform the algorithm and display the computation time.
23     initial_timestamp = perf_counter()
24     make_change(coins, money_to_return)
25     final_timestamp = perf_counter()
26     elapsed_time = final_timestamp - initial_timestamp
27
28     print(f'\nExecution time: {elapsed_time:0.7f} seconds.')
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
julien@5580-1:~/Documents/semester_at_the_university/master/s9/mobility_and_smart_cities/canalda$ /usr/bin/python3 /home/julien/Documents/se
da/tp/make_change_the_best.py
For the best solution, we need:

2 coins of 5.
1 coins of 2.
1 coins of 0.2.
1 coins of 0.1.

Execution time: 0.0000535 seconds.
```



results



Ishan Gautam a répondu à Ishan Gautam 25/11/2020

better results for viewing

(modifié)

```
5.0 -> 0 times  
2.0 -> 1 times  
1.0 -> 0 times  
0.5 -> 7 times  
0.2 -> 11 times  
0.1 -> 4 times  
0.05 -> 85.0 times
```

```
result is  
5.0 -> 0 times  
2.0 -> 1 times  
1.0 -> 0 times  
0.5 -> 7 times  
0.2 -> 11 times  
0.1 -> 5 times  
0.05 -> 83.0 times
```

```
result is  
5.0 -> 0 times  
2.0 -> 1 times  
1.0 -> 0 times  
0.5 -> 7 times  
0.2 -> 11 times  
0.1 -> 6 times  
0.05 -> 81.0 times
```

```
result is  
5.0 -> 0 times  
2.0 -> 1 times  
1.0 -> 0 times  
0.5 -> 7 times  
0.2 -> 11 times  
0.1 -> 7 times  
0.05 -> 79.0 times
```

```
result is  
5.0 -> 0 times  
2.0 -> 1 times  
1.0 -> 0 times  
0.5 -> 7 times  
0.2 -> 11 times  
0.1 -> 8 times  
0.05 -> 77.0 times
```

```
result is  
5.0 -> 0 times  
2.0 -> 1 times  
1.0 -> 0 times  
0.5 -> 7 times  
0.2 -> 11 times  
0.1 -> 13 times  
0.05 -> 67.0 times
```

```
result is  
5.0 -> 0 times  
2.0 -> 1 times  
1.0 -> 0 times  
0.5 -> 7 times  
0.2 -> 11 times  
0.1 -> 14 times  
0.05 -> 65.0 times
```

```
result is  
5.0 -> 0 times  
2.0 -> 1 times  
1.0 -> 0 times  
0.5 -> 7 times  
0.2 -> 11 times  
0.1 -> 15 times  
0.05 -> 63.0 times
```

```
result is  
5.0 -> 0 times  
2.0 -> 1 times  
1.0 -> 0 times  
0.5 -> 7 times  
0.2 -> 11 times  
0.1 -> 16 times  
0.05 -> 61.0 times
```